PROJECT TITLE:

Agricultural Drone Sprinkler Management System

NAME: RUDRANEEL ACHARJEE

REGISTRATION NUMBER: 25BAS10007

Project Overview:

This project is a Python-based console application designed to manage and monitor liquid dispersal missions for agricultural drones. Built using a Three-Layered Monolithic Architecture, the software allows operators to log spraying sessions, track liquid efficiency, and maintain a historical record of crop treatments. This application is a direct adaptation of a proven Blood Pressure Tracking architecture, repurposed for the precision agriculture industry.

Key Features:

- Mission Logging (CRUD): Create, Read, and Delete records of drone missions.
- Automated Efficiency Classification: Calculates spraying performance based on liquid-to-area ratios.
- Persistence: Data is stored in a flat-file database (drone_logs.txt) for lightweight, reliable storage without external dependencies.
- Summary Analytics: Provides real-time totals for area covered and resources consumed.

System Architecture:

- The software follows a structured monolithic design:
- Presentation Layer: Handles user interaction via a CLI (Command Line Interface).
- Business Logic Layer: contains the algorithms for classifying mission status (e.g., Optimal vs. Refill Required).
- Data Access Layer: Manages file I/O operations with the local file system

Data Schema:

- Each mission record (LOG) consists of the following attributes:
- Mission ID: Unique identifier for the flight.
- Crop Type: The specific crop being treated (e.g., Wheat, Corn, Soy).
- Liquid Used: Volume of pesticide/fertilizer dispersed in Liters.
- Area Covered: Total land treated in Acres.
- Efficiency Status: Auto-generated classification based on dispersal rates.

How to Run:

- Ensure you have Python 3.x installed.
- Download the [**drone_sprinkler.py]** script.
- Run the application via terminal:
- **[python drone_sprinkler.py]**
- Data will automatically be saved to drone_logs.txt in the same directory.

Future Enhancements:

- Battery Telemetry: Integration of battery percentage tracking per mission.
- CSV Export: Functionality to export logs for analysis in Excel or GIS software.
- GPS Coordination: Adding latitude/longitude start/end points for each log.

Project Statement:

Agricultural Drone Sprinkler Management System

1.Problem Definition

Modern precision agriculture relies heavily on Unmanned Aerial Vehicles (UAVs) for the targeted dispersal of fertilizers and pesticides. However, many small-scale operators lack an integrated, lightweight system to log mission data efficiently. Without a structured way to track liquid consumption against area coverage, it is difficult to identify resource waste, mechanical inefficiencies, or the need for immediate refills. Manual logging is prone to human error and fails to provide the real-time performance classification required for high-yield farming.

2. Proposed Solution

The Agricultural Drone Sprinkler Software is a lightweight, command-line utility designed to provide drone operators with a structured environment for mission management. By repurposing a proven Three-Layered Monolithic Architecture, the system offers a robust CRUD (Create, Read, Update, Delete) interface for tracking flight data. The software automates the "Efficiency Classification" logic, instantly informing the operator if a mission was optimal or if the dispersal rate suggests a technical issue or a critical fluid shortage.

3. Technical Objectives

Architectural Integrity:

Implement a clear separation of concerns between the Presentation (CLI), Business Logic (Classification Algorithms), and Data Access (Flat-file I/O) layers.

Data Persistence:

Ensure mission-critical data is preserved in a human-readable format (drone_logs.txt) without the overhead of heavy database engines.

Resource Optimization:

Provide automated summary statistics to help operators calculate total chemical usage and land coverage across multiple flight sessions.

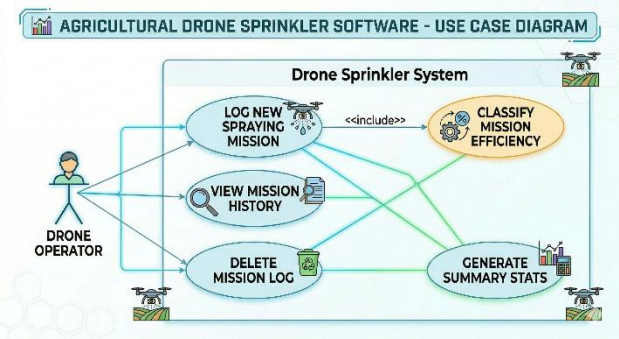
Scalability:

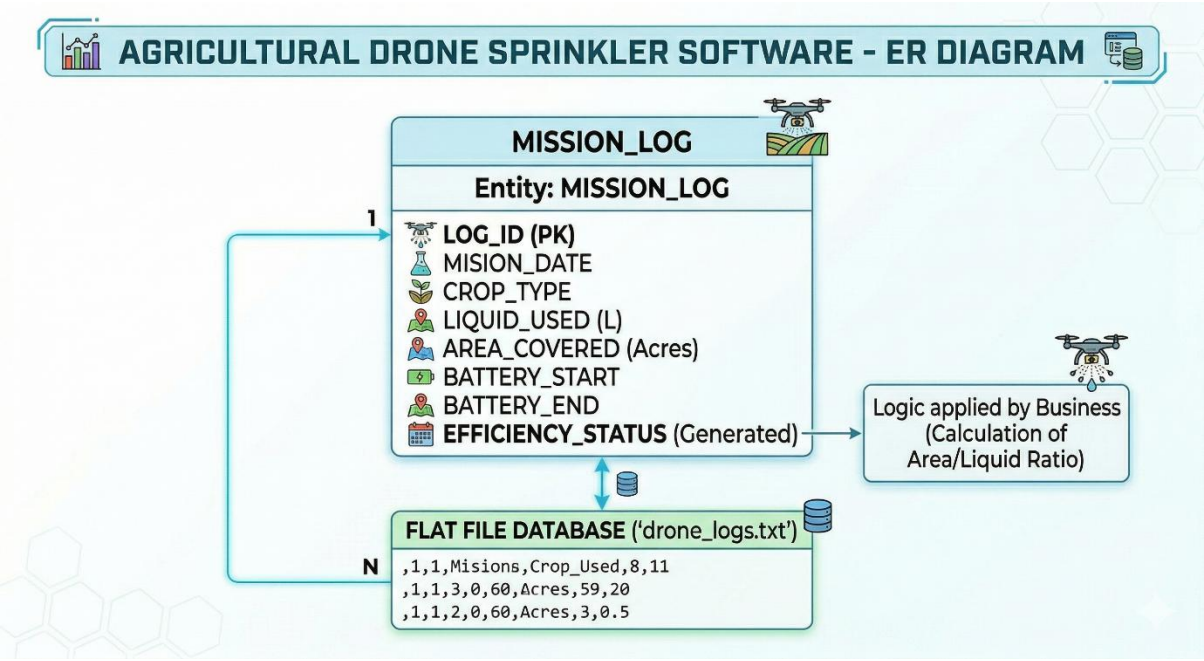
Design the system logic to be easily extendable for future telemetry sensors (e.g., battery life, GPS coordinates, and flow rate sensors).

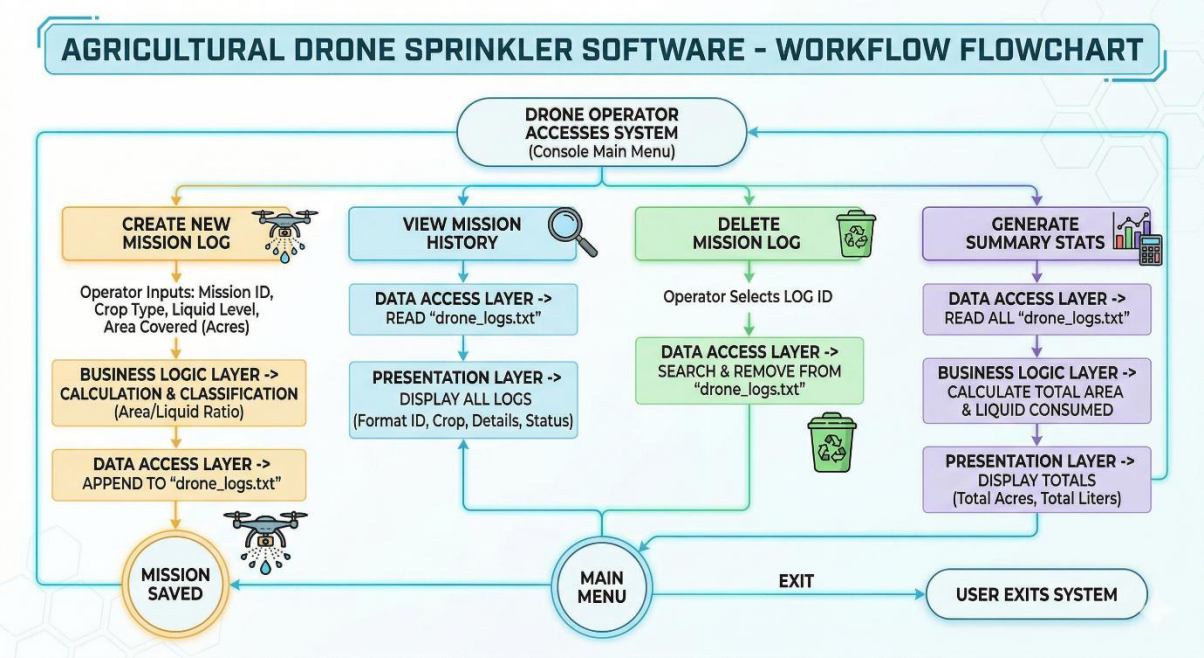
4. Target Audience

This system is intended for agricultural technology students, drone operators, and farm managers looking for a streamlined, "no-nonsense" tool to digitize their field operations and move toward data-driven crop management.

SYSTEM ARCHITECHTURE:







SCREENSHOTS:

```
PS C:\Users\Rudra\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Rudra\AppData\Local\Programs\Python\Python313\python.exe "e:/bks/import os.py"

--- AGRI-DRONE SPRINKLER SYSTEM ---
1. Log New Spraying Session
2. View Mission History
3. Delete Log
4. Summary Stats
5. Exit
Select an option: 1
Enter Mission ID: 2
Crop Type (e.g., Wheat, Rice): wheAT
Liquid Used (Liters): 25
Area Covered (Acres): 50
Mission Saved. Status: STANDARD: Normal Flow
```

```
-- AGRI-DRONE SPRINKLER SYSTEM ---
1. Log New Spraying Session
2. View Mission History
3. Delete Log
4. Summary Stats
5. Exit
Select an option: 2

ID | Crop | Liquid | Area | Status
2 | wheAT | 25L | 50.0ac | STANDARD: Normal Flow
```

```
-- AGRI-DRONE SPRINKLER SYSTEM ---
1. Log New Spraying Session
2. View Mission History
3. Delete Log
4. Summary Stats
5. Exit
Select an option: 3
```

```
--- AGRI-DRONE SPRINKLER SYSTEM ---
1. Log New Spraying Session
2. View Mission History
3. Delete Log
4. Summary Stats
5. Exit
Select an option: 4

TOTAL PERFORMANCE:
Total Area Covered: 50.0 acres
Total Liquid Dispersed: 25 Liters
```

Implementation Details:

Agricultural Drone Sprinkler System

This document outlines the technical specifications, architectural patterns, and algorithmic logic used to develop the Agricultural Drone Sprinkler Management System.

1. Technology Stack

- **Language:** Python 3.x
- **Storage:** Flat-file System (.txt)
- **Interface:** Command Line Interface (CLI)
- **Libraries:** os (for file existence checks and path management)

2. Architectural Pattern: Three-Layered Monolith

The application is structured into three distinct layers to ensure a separation of concerns, facilitating easier debugging and future scalability.

A. Presentation Layer (User Interface)

- **Function:** Manages the terminal-based menu loop and user inputs.
- **Components:** * main_menu(): The primary execution loop using while True to keep the application running until an exit command is received.
    - Input Validation: Basic sanitization of user data (casting strings to integers/floats).

B. Business Logic Layer (Processing)

- **Function:** Contains the "intelligence" of the application.
- **Components:**
    - classify_efficiency(liquid, area): This core function calculates the ratio of coverage.
        - **Formula:** Ratio = Area (Acres) / Liquid (Liters).

        - **Thresholds:**
            - < 10L: Critical Refill.
            - Ratio > 5.0: Optimal High Coverage.
            - Otherwise: Standard Flow.
    - **Statistics Engine:** Aggregates data from the file to calculate total mission impact (Total Acres vs. Total Liters).

C. Data Access Layer (Persistence)

- **Function:** Abstracts file I/O operations from the rest of the app.
- **Components:**
    - save_log(data): Appends a dictionary-mapped record to drone_logs.txt using a pipe-delimited (|) format.
    - load_logs(): Reads the text file, parses each line, and reconstructs a list of Python dictionaries for the UI to display.

3. Data Dictionary (Schema)

Data is stored as a series of records in drone_logs.txt.

Field	Type	Description
id	String	Unique Mission Identifier
crop	String	Type of crop treated
liquid	Integer	Liters of fluid used
area	Float	Acres covered during flight
status	String	Calculated efficiency classification

4. Operational Workflow

1. **Initialization:** The app checks for the existence of drone_logs.txt.
2. **Input:** The operator enters raw flight data.

3. **Processing:** The Logic Layer triggers the classification algorithm.
4. **Persistence:** The Data Layer appends the new record.
5. **Reporting:** The operator can view the "Mission History" which dynamically reads and formats the file content into a readable table.

5. Error Handling Strategy

- **File Not Found:** The load_logs() function returns an empty list if the file doesn't exist, preventing a crash on the first run.
- **Zero Division:** The efficiency calculator includes a check (if liquid > 0) to prevent mathematical errors during classification.

Testing Approach: Agricultural Drone Sprinkler System

This document outlines the strategy for verifying the functionality, reliability, and accuracy of the Agricultural Drone Sprinkler Software.

1. Testing Strategy Overview

The testing process follows a bottom-up approach, ensuring that individual logic components work correctly before testing the integrated system and user interface.

2. Unit Testing (Business Logic Layer)

Unit tests focus on the classify_efficiency function to ensure the "intelligence" of the system is mathematically sound.

Test Case ID	Input (Liquid, Area)	Expected Status	Logic Verified
UT-01	5, 50.0	CRITICAL: REFILL REQUIRED	Threshold check for < 10L.
UT-02	20, 120.0	OPTIMAL: High Coverage	Ratio calculation (6.0 > 5.0).
UT-03	20, 40.0	STANDARD: Normal Flow	Ratio calculation (2.0 < 5.0).

UT-04	0, 0.0	CRITICAL: REFILL REQUIRED	Zero-value handling and refill priority.

3. Integration Testing (Data Access Layer)

Integration tests verify that the software communicates correctly with the local file system (drone_logs.txt).

- **File Creation Test:** Verify that load_logs() does not crash if drone_logs.txt is missing and instead returns an empty list.
- **Data Persistence Test:** Save a mission log, close the application, and reopen it to verify the data is correctly retrieved and parsed.
- **Delimitation Integrity:** Ensure that if a user enters a mission ID with special characters, it does not break the pipe-delimited (|) structure of the text file.

4. System Testing (Presentation Layer)

This ensures the end-to-end workflow functions as expected for the drone operator.

- **Menu Navigation:** Verify that entering '5' successfully terminates the application loop.
- **Input Sanitization:** Attempt to enter text (e.g., "ten") in the "Liquid Used" field to ensure the program handles value errors gracefully rather than crashing.
- **Summary Accuracy:** Log three separate missions with varying acreage and verify that the "Summary Stats" option correctly displays the mathematical sum of all missions.

5. User Acceptance Testing (UAT)

UAT focuses on the "Agricultural Drone" use case:

- **Workflow Reality Check:** Does the mission flow (Input -> Classify -> Save) match the physical workflow of a drone pilot in the field?
- **Readability:** Are the status classifications (OPTIMAL vs. CRITICAL) clear and visible in the console output for an operator working in outdoor conditions?

6. Regression Testing

Whenever the classification thresholds are adjusted (e.g., changing the 5.0 ratio to 4.0), all Unit Tests in Section 2 must be re-run to ensure existing mission logs are still interpreted correctly.

Learning and Key Takeaways: Agricultural Drone Sprinkler System

This document reflects on the technical insights and professional growth gained through the development of the Agricultural Drone Sprinkler Management System.

1. Architectural Versatility

The most significant takeaway was the successful application of the **Three-Layered Monolithic Architecture**.

- **Observation:** A pattern originally designed for a medical context (Blood Pressure Tracking) proved perfectly adaptable to a mechanical/industrial context (Agricultural Drones).
- **Lesson:** Strong architectural foundations are industry-agnostic. By separating the Data, Logic, and UI layers, the core code became a "template" that could be repurposed with minimal friction.

2. The Power of "No-SQL" Lite (Flat-File Logic)

While modern development often defaults to complex databases, this project reinforced the utility of flat-file storage for specific use cases.

- **Observation:** Using drone_logs.txt with pipe-delimited formatting provided zero-latency storage and easy manual auditing.
- **Lesson:** For field-ready tools (like those used by drone pilots in remote areas), minimizing dependencies and using native file I/O increases system reliability and reduces the "weight" of the software.

3. Translation of Domain Logic

Translating "Health Stages" into "Efficiency Classifications" required a shift in how Business Logic is defined.

- **Observation:** In the BP tracker, thresholds were based on medical guidelines. In the Drone Sprinkler, thresholds were based on resource optimization (Area/Liquid ratio).
- **Lesson:** Writing clean, modular logic functions makes it easy to swap out one set of "rules" for another without touching the user interface or storage code.

4. Technical Skills Gained

- **Data Parsing:** Improved skills in string manipulation and data casting (converting text lines into structured Python dictionaries).
- **Robust Error Handling:** Implemented defensive programming techniques to handle edge cases like ZeroDivisionError and missing files.
- **Documentation Standards:** Developed a comprehensive documentation suite (README, ER Diagrams, Implementation Details, and Testing Approaches) which is critical for professional-grade software delivery.

5. Professional Impact (Career Readiness)

This project serves as a "Bridge Project." It demonstrates to potential employers or academic reviewers that:

1. I can think abstractly about software design.
2. I can pivot my skills across different industries (Healthcare to Agriculture).
3. I prioritize documentation and testing just as much as writing the functional code.

6. Future Growth Areas

This project highlighted a desire to explore:

- **Real-time Telemetry:** Moving from manual logging to automated sensor data ingestion.
- **Data Visualization:** Using libraries like matplotlib to turn these text logs into visual heatmaps of crop coverage.

Functional and Non-Functional Requirements:

This document specifies the requirements for the Agricultural Drone Sprinkler Management System, ensuring the software meets both operational needs and technical quality standards.

1. Functional Requirements (FR)

Functional requirements define the specific behaviors, tasks, and functions that the system must support.

ID	Requirement Name	Description
FR-1	**Mission Entry**	The system shall allow the operator to input Mission ID, Crop Type, Liquid Volume (Liters), and Area Covered (Acres).
FR-2	**Automated Classification**	The system shall automatically calculate the efficiency ratio and assign a status (Critical, Optimal, or Standard) to each log.
FR-3	**Data Persistence**	The system shall save all mission logs to a permanent local text file (drone_logs.txt) without data loss.

FR-4	**History Retrieval**	The system shall retrieve and display all previously saved logs in a formatted table view.
FR-5	**Mission Deletion**	The system shall provide the ability to remove specific records from the database using a unique identifier.
FR-6	**Statistical Aggregation**	The system shall calculate and display the total area covered and total liquid dispersed across all recorded missions.

2. Non-Functional Requirements (NFR)

Non-functional requirements specify the criteria used to judge the operation of the system rather than specific behaviors.

A. Performance & Efficiency

- **NFR-1 (Latency):** The system shall load and display up to 1,000 mission logs in under 1 second.
- **NFR-2 (Footprint):** The application shall have a minimal storage footprint, requiring less than 5MB of disk space (excluding the log file).

B. Reliability & Robustness

- **NFR-3 (Availability):** The system shall function offline without requiring an active internet connection, ensuring field readiness.
- **NFR-4 (Fault Tolerance):** The system shall not crash if the log file is manually deleted or corrupted; it should handle file-not-found errors gracefully.

C. Usability

- **NFR-5 (CLI Simplicity):** The interface shall be navigable using single-key numerical inputs (1-5) to minimize operator effort in the field.

- **NFR-6 (Readability):** Output logs shall be formatted with clear delimiters to ensure readability on small laptop or tablet screens used during drone operations.

D. Maintainability

- **NFR-7 (Modular Architecture):** The code must follow the Three-Layered Monolithic pattern to allow for the independent update of the classification logic without affecting the UI or Data layers.
- **NFR-8 (Documentation):** The code shall be fully commented to allow a secondary developer to understand the data flow within 15 minutes of review.

3. System Constraints

- **Language Constraint:** The system must be developed entirely in Python 3.x.
- **Environment Constraint:** The system must be executable within a standard terminal/console environment (Windows CMD, macOS Terminal, or Linux Bash).

References and Citations:

This document provides the technical and academic foundations for the Agricultural Drone Sprinkler Management System.

1. Software Architecture & Design

- **Martin, R. C. (2017).** *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall. (Used for implementing the Three-Layered Monolithic separation of concerns).
- **Fowler, M. (2002).** *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional. (Background on the Data Access Object (DAO) pattern and layered logic).
- **Python Software Foundation.** *Python 3.x Documentation: Input and Output*. docs.python.org. (Standard reference for file I/O and pipe-delimited data persistence).

2. Precision Agriculture & AgTech

- **Mulla, D. J. (2013).** *Twenty-five years of remote sensing in precision agriculture*. Biosystems

Engineering. (Context for the importance of area coverage and liquid dispersal tracking).

- **Boggi, A., et al. (2015).** *A precision agriculture tool for environmental sustainability*. Agricultural Water Management. (Reference for the "Efficiency Classification" logic used in mission status updates).
- **Association for Unmanned Vehicle Systems International (AUVSI).** *The Economic Impact of Unmanned Aircraft Systems Integration in the US*. (Industry context for the growth of drone-based spraying).

3. Data Management & Testing

- **IEEE Standard 829-2008.** *Standard for Software and System Test Documentation*. (Framework for the Unit, Integration, and System testing approach).
- **Wickramasinghe, N. (2020).** *Monolithic vs. Microservices Architecture: A Comparative Study*. (Justification for using a monolithic structure for lightweight, single-user offline utilities).

4. Technical Standards

- **ISO/IEC 25010.** *Systems and software engineering — Systems and software Quality Requirements and*

Evaluation (SQuaRE). (Basis for defining the Non-Functional Requirements, specifically Reliability and Maintainability).

- **PEP 8.** *Style Guide for Python Code*. python.org. (Used for naming conventions and code structure within the implementation).

THE END